

152. Un raccourci à toi : écrire poserDevant() soi-même

poserDevant() de la console tient en deux lignes : on l'écrit soi-même, dans outils.cpp. Et l'on découvre pourquoi celle de la console est plus légère.

1. Ce qu'il faut comprendre

Une fonction « raccourci », c'est juste un nom donné à des gestes qu'on sait déjà faire. poserDevant() (le 110.10) pose une tuile, puis la teint | DEVANT. Ici, on l'écrit **soi-même**, dans outils.cpp, comme bande() et pile().

Lis-la : quatre paramètres, deux lignes. poser(colonne, ligne, tuile) pose la tuile ; teindre(colonne, ligne, palette | DEVANT) choisit sa palette et la fait passer devant les lutins.

Elle porte le même nom que celle de la console : c'est donc **la tienne** qui sert (le 15.7). Le programme n'écrit pas #include <poserDevant> : il n'en a pas besoin. Il inclut poser et teindre, dont TA fonction se sert.

La différence de poids : ta version est une vraie fonction. À chaque appel, le programme range quatre nombres, saute dans la fonction, et poser() et teindre() ne savent plus rien d'avance (la case, la tuile) : elles calculent tout pendant le jeu. Celle de la console, elle, n'est **pas une vraie fonction** : le compilateur **recopie ses deux lignes à la place de l'appel**, avec les vrais nombres. Le résultat est le même à l'écran ; la cartouche de la console est plus petite.

Alors, la tienne ne sert à rien ? Si : elle est à toi, tu peux la changer. Une version qui pose **deux** cases d'un coup (un buisson de 2 de large), une autre qui choisit toute seule la palette... La console ne fera jamais ce que ton jeu a en tête.

À toi : ajoute dans outils.cpp un buissonDevant(colonne, ligne) qui appelle quatre fois poserDevant() pour un buisson de 2 × 2 cases.

Les fonctions de cette leçon — et le tuto de chacune

- #include <Tuile> → [0.107.2. Tuile — dessiner une case](#)
- #include <couleurFond> → [0.88.3. La fonction couleurFond\(\) — une couleur du décor](#)
- #include <couleurLutin> → [0.107.4. La fonction couleurLutin\(\) — une couleur des personnages](#)
- #include <sprite> → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- #include <teindreLutin> → [0.107.5. La fonction teindreLutin\(\) — un lutin dans une palette](#)
- #include <poser> → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- #include <teindre> → [0.89.1. La fonction teindre\(\) — une case dans une palette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1187 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l’a demandé — et ce qu’il a laissé, parce que rien ne l’appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l’allumage – la mise en route de la console
poserDevant()	40 o	← main() – une fonction fournie par le langage
main()	224 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l’image suivante
AttendreVBlank	16 o	← démarrage – attend que l’écran soit libre
AdresseCase	10 o	← poser(), teindre() – trouve l’adresse d’une case de l’écran
RangerLutins	11 o	← démarrage – range les 40 lutins hors de l’écran
InstallerTransfert	15 o	← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l’écran
Tuiles	736 o	← CopierTuiles – la police entière, et 2 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert – la petite routine qui copie les lutins

Lettres gravées : toutes (poser()) reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

sprite()	x 2	46 o sur place, dans main()
couleurFond()	x 3	36 o sur place, dans main()
teindreLutin()	x 2	32 o sur place, dans main()
teindre()	x 1	26 o sur place, dans poserDevant() + AdresseCase (10 o)
couleurLutin()	x 2	24 o sur place, dans main()
poser()	x 1	13 o sur place, dans poserDevant() + AdresseCase (10 o)
image()	x 1	6 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

poserDevant()	x 4	76 o sur place, dans main()
---------------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// Le buisson DEVANT, avec MA poserDevant(), écrite dans outils.cpp.  
// Pas de « #include <poserDevant> » : ma fonction vient d'outils.cpp.  
  
#include <Tuile>           // un dessin de 8 × 8 pixels  
#include <couleurFond>     // choisit une couleur d’une palette du fond  
#include <couleurLutin>    // choisit une couleur d’une palette des lutins  
#include <sprite>          // place un lutin de 8 × 8 au pixel près  
#include <teindreLutin>    // met un lutin dans une palette  
#include "outils.cpp"      // MA poserDevant()
```

2. UN DESSIN — BUISSON

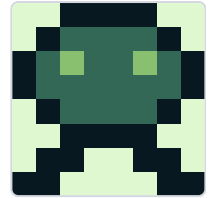
```
Tuile BUISSON = {           // Une touffe de feuilles : AUCUN 0, elle est toute pleine.  
    "11222211",           // (des coins en 0 laisseraient un trou là où deux  
    "12233221",           // touffes se touchent : on y verrait le héros)  
    "22333322",  
    "23333332",  
    "23333332",  
    "22333322",  
    "12233221",  
    "11222211",  
};
```



à l’écran

3. UN DESSIN — HEROS

```
Tuile HEROS = {                                // 0 transparent, 1 clair, 2 moyen, 3 contour.
    "00333300",
    "03222230",
    "32122123",
    "32222223",
    "03222230",
    "00333300",
    "03300330",
    "3300033",
};
```



à l'écran

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    couleurFond(2, 1, 16, 30, 8); // palette 2 : les verts du buisson
    couleurFond(2, 2, 6, 22, 4);
    couleurFond(2, 3, 0, 10, 2);
    couleurLutin(0, 2, 31, 4, 2); // le héros : rouge
    couleurLutin(0, 3, 10, 0, 0);

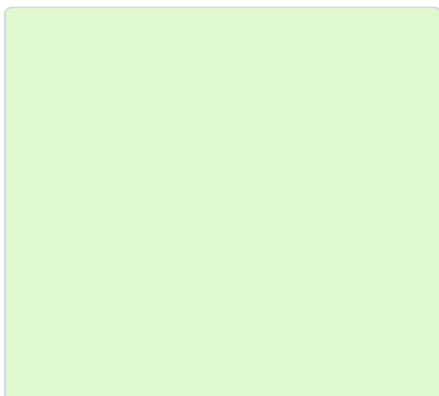
    // Un buisson de 2 x 2 cases, colonnes 12-13, lignes 8-9 : quatre appels.
    poserDevant(12, 8, BUISSON, 2);
    poserDevant(13, 8, BUISSON, 2);
    poserDevant(12, 9, BUISSON, 2);
    poserDevant(13, 9, BUISSON, 2);

    sprite(0, 96, 68, HEROS); // le héros, dans le buisson : caché
    teindreLutin(0, 0);
    sprite(1, 40, 68, HEROS); // un autre, à découvert : visible
    teindreLutin(1, 0);

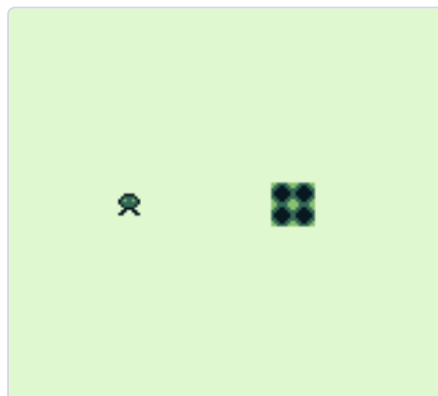
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un buisson vert, et un héros rouge à gauche. Le second héros, dans le buisson, est caché.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ `ma poserDevant()` a teint la case (13, 9) : palette 2 et DEVANT (130)

✓ dans le buisson, on voit le buisson (vert) (6/22/4)

✓ à découvert, on voit le héros (rouge) (31/4/2)

Ensuite : **2.01. Le rond** — série 2 : Les formes géométriques. · [gameboy3](#) — écrire une cartouche Game Boy en C++